

# **Competitive Programming**

## **Section 40: Data Structures Mixed Contest**

Competitive Programming Course

2026-07-10

# Table of contents

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Combining data-structure ideas</b>                             | <b>3</b>  |
| <b>2</b>  | <b>First-fit with a segment tree</b>                              | <b>4</b>  |
| <b>3</b>  | <b>Removing by order</b>                                          | <b>5</b>  |
| <b>4</b>  | <b>Mo's algorithm</b>                                             | <b>6</b>  |
| <b>5</b>  | <b>Choosing between similar tools</b>                             | <b>7</b>  |
| <b>6</b>  | <b>Mixed contest solving order</b>                                | <b>8</b>  |
| <b>7</b>  | <b>Contest checklist</b>                                          | <b>9</b>  |
| <b>8</b>  | <b>Local exercises</b>                                            | <b>10</b> |
| <b>9</b>  | <b>How to use the checker</b>                                     | <b>11</b> |
| <b>10</b> | <b>Operational traces and invariants</b>                          | <b>12</b> |
| 10.1      | First feasible hotel: descend by subtree maximum . . . . .        | 12        |
| 10.2      | Remove by current order: Fenwick counts alive positions . . . . . | 12        |
| 10.3      | Mo's algorithm: move a window, not every query . . . . .          | 12        |
| <b>11</b> | <b>Language notes and adversarial practice</b>                    | <b>13</b> |

# 1 Combining data-structure ideas

Contest data-structure problems rarely say “use a segment tree” or “use a Fenwick tree.” They describe operations. Your job is to map operations to the smallest structure that supports them.

Read operations as contracts:

```
point update + prefix/range sum      -> Fenwick
point update + arbitrary range min    -> segment tree
range update + range query            -> lazy segment tree or two Fenwicks
static range idempotent query         -> sparse table
offline add/remove endpoints          -> Mo's algorithm
find first position satisfying query  -> segment tree descent or Fenwick search
```

## 2 First-fit with a segment tree

If every hotel has remaining capacity and each group wants the first hotel with capacity at least  $x$ , store maximum capacity in each segment tree node. If the root maximum is smaller than  $x$ , no hotel works. Otherwise descend:

```
if left_child.max >= x: go left
else: go right
```

This is a binary search guided by aggregate information.

Example:

```
capacities = [2, 8, 4, 6]
group = 5
```

The root maximum is 8, so some hotel works. Check the left half [1,2]:

```
max([2,8]) = 8 >= 5
```

Go left. Then check hotel 1:

```
2 < 5
```

So go to hotel 2, subtract the group, and update capacity  $8 \rightarrow 3$ .

### 3 Removing by order

If elements are removed from a list and queries ask for the k-th alive element, store 1 for alive positions and 0 for removed positions. A Fenwick tree can find the smallest index with prefix sum at least k by binary lifting on the Fenwick structure.

Example:

```
alive = [1, 1, 0, 1, 1]
prefix sums = [1, 2, 2, 3, 4]
```

The 3-rd alive element is the first position with prefix sum at least 3, which is position 4.

After removing it:

```
alive = [1, 1, 0, 0, 1]
```

## 4 Mo's algorithm

Mo's algorithm answers offline range queries by ordering them so the current interval changes slowly. Maintain a window  $[L,R]$  and four operations:

```
add_left, add_right, remove_left, remove_right
```

For distinct count, a frequency table plus a **distinct** counter is enough. Mo is useful when updates are absent and adding/removing one endpoint is cheap.

Mo's ordering groups queries by block of  $L$ , then sorts by  $R$  inside each block:

```
block = L // sqrt(n)
sort by (block, R)
```

This keeps the window from jumping too far on average. The algorithm is not online: you must know all queries before answering them.

## 5 Choosing between similar tools

Fenwick and segment trees overlap, but the decision is usually simple:

```
Need prefix sums or order statistics? Fenwick is shorter.  
Need min/max over arbitrary intervals? Segment tree.  
Need first index whose segment aggregate satisfies a condition? Segment tree.  
Need k-th alive by prefix counts? Fenwick binary lifting.
```

Mo's algorithm is different: it is not a tree. It trades online answers for an offline ordering that makes add/remove operations cheap. Use it when the query function is easy to update after moving one endpoint but hard to merge from two children.

## 6 Mixed contest solving order

When a section mixes structures, do not pick the first familiar tool. Write the operation table:

```
update type:  
query type:  
online/offline:  
need first position or only value:  
can answers be reordered:
```

For Hotel Queries, “first position” is the clue. For List Removals, “k-th alive” is the clue. For Mo Distinct Queries, “offline range queries with cheap add/remove” is the clue.

## 7 Contest checklist

Are operations online or can queries be reordered?

Is the query about prefix/rank/order? Consider Fenwick.

Is the query about arbitrary intervals? Consider segment tree.

Is the array static and queries many? Consider sparse table or Mo.

## 8 Local exercises

- A. Hotel Queries: For each group size, place it in the first hotel with enough remaining capacity, subtract that size, and print the hotel index or 0.
- B. List Removals: Repeatedly remove and print the  $k$ -th currently alive element.
- C. Mo Distinct Queries: For each offline range query, print the number of distinct values in the range.

## 9 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/40_data_structures_mixed_contest/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.  
# Exercises

## 10 Operational traces and invariants

### 10.1 First feasible hotel: descend by subtree maximum

Each segment-tree node stores the maximum remaining capacity in its interval. For request 4 and capacities  $[3, 1, 4, 1, 5]$ , the root is feasible. Descend left whenever the left child's maximum is at least four; otherwise the first feasible hotel must be in the right child. The descent reaches index three, then its capacity becomes zero and every ancestor maximum is recomputed.

The invariant is: a node value equals the maximum capacity of all leaves below it. If a left child is feasible, choosing it preserves “first index”; if it is not, no index in that left interval can work. This proves both the search and the  $O(\log n)$  update.

### 10.2 Remove by current order: Fenwick counts alive positions

For values  $[10, 20, 30, 40, 50]$ , initialize counts  $[1, 1, 1, 1, 1]$ . Removing current position two asks for the smallest original index with prefix count at least two: it is index two, output 20, then add  $-1$  there. The alive counts become  $[1, 0, 1, 1, 1]$ ; the next current position three resolves to original index four, output 40.

Fenwick `kth(k)` binary-lifts through its internal partial sums. Its invariant is that `pos` has cumulative count strictly below the remaining `k`; after the largest possible steps, `pos+1` is the first prefix reaching the original rank.

### 10.3 Mo's algorithm: move a window, not every query

Offline range queries can be ordered by left block and a zig-zag right endpoint. For  $[1, 2, 1, 3, 2, 4]$ , start with empty  $[0, -1]$ ; to answer  $[1, 3]$ , add values  $1, 2, 1$ , producing frequencies  $\{1:2, 2:1\}$  and distinct count two. Moving to  $[2, 5]$  removes the first 1, then adds  $3, 2$ , yielding three distinct values.

The maintained invariant is that `freq[v]` counts exactly the active window; `distinct` counts values with positive frequency. Compression is required when values are large or negative. A block size near  $\sqrt{n}$  balances left and right movement, giving about  $O((n+q)\sqrt{n})$  moves.

# 11 Language notes and adversarial practice

**C++.** Store hotel capacities and sums in `long long`; Fenwick frequencies fit in `int`. Convert Mo queries from one-based to zero-based exactly once. Use a parity-reversed right ordering to reduce constant factors.

**Python.** These local algorithms are viable with iterative arrays, buffered input, and a compact frequency list after compression. Avoid a recursive segment tree and avoid `list.pop(k)`, which is linear per removal.

| Exercise           | Edge case to test                                     |
|--------------------|-------------------------------------------------------|
| A: first feasible  | no capacity reaches request, answer 0                 |
| B: order statistic | remove first, last, then the only survivor            |
| C: distinct        | all equal, all distinct, and repeated equal endpoints |

Before coding, state the stored aggregate: maximum for A, alive count for B, and frequency plus distinct count for C. Run `CP_TARGET=solution python3 sections/40_data_structures_mixed_contest/ch`. Stress A with a linear first-fit oracle, B with a mutable short list, and C with `len(set(a[1:r+1]))` on arrays of length at most 30.

Choose the structure from the operation: A needs first position with enough capacity, B needs k-th alive order statistics, and C benefits from offline range order. Test updates at both ends and restore query order explicitly.